

CineVerse

Phase 6 — DB-Only Concurrency Baseline

Engineering Benchmark Record & Architectural Decision Baseline
Inventory Service — Seat Reservation Engine

1. Purpose of This Document

This document records the empirical DB-only concurrency baseline for CineVerse before introducing Redis. It serves as the engineering reference point for evaluating whether Redis provides measurable value as a contention-reduction layer under highly concurrent seat reservation traffic.

The benchmark is intentionally based on the CineVerse principle that performance must be measured rather than assumed, while correctness remains the primary requirement. CineVerse requires exactly one successful reservation and zero double bookings when 10,000 concurrent requests target the same seat.

2. Test Scenario

Item	Value
Service	Inventory Service — Seat Reservation Engine
Persistence	PostgreSQL 18
Framework	Spring Boot 3.x
Persistence Layer	Spring Data JPA / Hibernate
Concurrency Harness	ExecutorService — 100 worker threads
Synchronization	CountDownLatch gate synchronization
Target Load	10,000 concurrent hold requests
Target Record	show_seat_id = 1L
Concurrency Mechanism	PostgreSQL atomic conditional UPDATE

3. Correctness Mechanism

The DB-only implementation uses an atomic conditional state transition. Only a request that observes AVAILABLE can change the seat to HELD.

```
UPDATE inventory_db.show_seats
SET status = 'HELD'
WHERE show_seat_id = :showSeatId
AND status = 'AVAILABLE';
```

The benchmark observed one successful update and 9,999 rejected attempts, with zero double bookings. This establishes PostgreSQL as the authoritative correctness boundary for the seat state.

4. Empirical DB-Only Baseline

Category	Metric	Recorded Value	Result
Correctness	Concurrent Requests	10,000	PASS
Correctness	Successful Holds	1	PASS
Correctness	Rejected Requests	9,999	PASS

Category	Metric	Recorded Value	Result
Correctness	Double Bookings	0	PASS
Latency	P50	10.46 ms	BASELINE
Latency	P95	87.66 ms	BASELINE
Latency	P99	157.82 ms	BASELINE
Throughput	Peak Processing Rate	5,166 req/sec	BASELINE
Throughput	Timed Processing Rate	4,400.13 req/sec	BASELINE
Duration	Peak Execution	1.936 sec	PASS
Duration	Timed Benchmark	2.272 sec	PASS

5. What the Baseline Demonstrates

- **Correctness:** PostgreSQL successfully prevents double booking under 10,000 concurrent attempts against the same seat.
- **Atomicity of the state transition:** The conditional UPDATE allows exactly one request to transition AVAILABLE → HELD.
- **Tail latency exists:** P99 latency is substantially higher than P50, showing that a small portion of requests experience significantly more delay under contention.
- **Database is handling every attempt:** In the DB-only architecture, competing requests reach the database before being rejected.

6. Architectural Problem Identified

The DB-only approach is correct, but it makes PostgreSQL the first contention point for every competing request. During a flash-sale style spike, many requests can compete for the same seat and database connection resources even though only one request can ultimately succeed.

The measured P50/P95/P99 latency distribution provides evidence that contention has a performance cost. The baseline therefore gives CineVerse a concrete comparison point for the next architecture rather than assuming Redis will improve performance.

7. Why Redis Is Being Introduced

Redis is not being introduced to replace PostgreSQL correctness. It is being introduced as a distributed lock / fast-fail contention layer. The intended architecture is:

```

10,000 concurrent requests
↓
Redis distributed lock / guard
↓
Only the lock owner proceeds to PostgreSQL
↓
PostgreSQL conditional UPDATE
↓
AVAILABLE → HELD

```

The expected benefit is to reject losing contenders in memory before they consume database resources. PostgreSQL remains the source of truth for the final seat state.

8. Redis Phase Hypothesis

The hypothesis to test is: Redis can reduce unnecessary database contention by preventing most competing requests from reaching PostgreSQL, while preserving the same correctness guarantee.

Important: this is a hypothesis, not a proven result. The Redis implementation must be benchmarked using the same 10,000-request workload and comparable test conditions.

9. Fair Comparison Criteria

Metric	DB Only	Redis + DB
Requests	10,000	10,000
Successful Holds	1	Must remain 1
Double Bookings	0	Must remain 0
P50	10.46 ms	Measure
P95	87.66 ms	Measure
P99	157.82 ms	Measure
Throughput	4,400–5,166 req/sec	Measure
DB contention/load	Measure baseline	Measure reduction

10. Decision Boundary

This document establishes the DB-only baseline. The Redis decision should be made only after the Redis implementation is benchmarked and its effects on latency, throughput, database load, and correctness are measured.

CineVerse's correctness invariant remains unchanged: Redis must never become the ultimate authority for whether a seat is successfully booked. If Redis fails, the system must retain a correctness-preserving fallback, even if performance becomes worse.

11. Phase 6 Status

Phase 6 Activity	Status
DB-only correctness test	COMPLETED
DB-only benchmark	COMPLETED
DB baseline recorded	COMPLETED
Redis distributed locking	NEXT
Redis + DB benchmark	PENDING
Redis failure test	PENDING
Fallback strategy	PENDING
Final DB vs Redis decision	PENDING

Baseline conclusion: PostgreSQL-only concurrency control is functionally correct. Redis is the next experiment, introduced specifically to reduce unnecessary contention before requests reach PostgreSQL—not to replace PostgreSQL as the correctness authority.